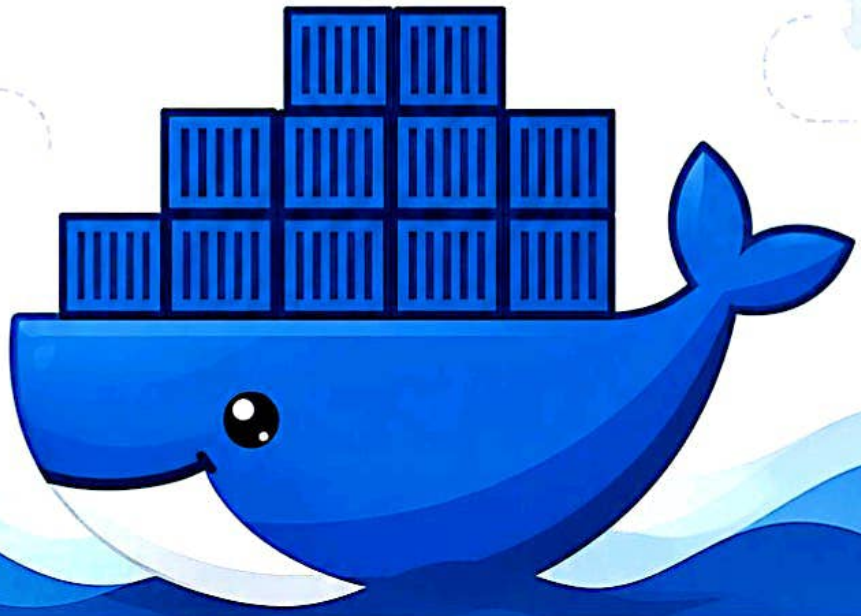
LEARN • BUILD • CONTAINERIZE • DEPLOY

DOCKER

BEGINNER TO ADVANCED

10-PAGE VISUAL GUIDE

A complete visual roadmap to master Docker
from fundamentals to real-world usage.



VISUAL GUIDE
Diagrams and concepts made easy.

COMMANDS
Essential Docker commands with examples.

PRACTICAL
Real-world examples and use cases.

WORKFLOWS
Step-by-step workflows and best practices.

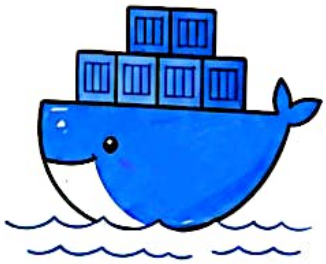
CAREER READY
Foundation for DevOps, Cloud, CI/CD & more.

-- -- @NeerajSingh-DevOps -- --

01 OF 10

DOCKER FUNDAMENTALS

The foundation of containerization



Build, Ship and Run Any App, Anywhere.

WHAT IS DOCKER?

Docker is an open-source platform that allows you to develop, ship and run applications inside containers.

It packages an application and all its dependencies together in a lightweight, portable and consistent unit.

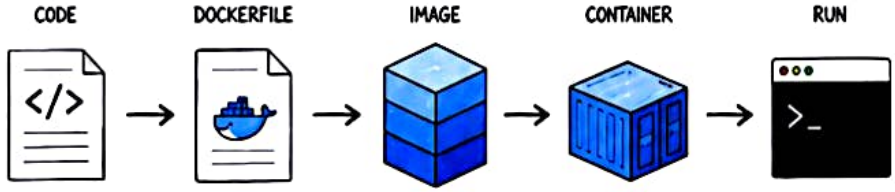
CONTAINER vs VM

VIRTUAL MACHINE	CONTAINER
Includes Guest OS	Shares Host OS Kernel
Heavyweight	Lightweight
Slower to start	Starts in seconds
Uses more RAM & Disk	Uses fewer resources
Hardware Virtualization	OS-level Virtualization

WHY DOCKER?

- Lightweight & Fast
- Consistent Across Environments
- Easy to Deploy & Scale
- Efficient Resource Usage
- Great for DevOps & CI/CD
- Runs Anywhere

KEY CONCEPT



COMMON USE CASES

- Web Applications
- Microservices
- Databases
- CI/CD Pipelines
- Development Environments
- Cloud & Hybrid Deployments

KEY TAKEAWAYS

- Docker packages your application and its dependencies into a container.
- Containers are lightweight, portable and consistent.
- Docker ensures "it works on my machine" everywhere.
- It is the backbone of modern DevOps and Cloud-native apps.

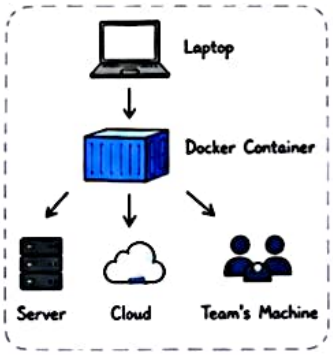
REAL WORLD EXAMPLE

You build a Flask app on your laptop. Using Docker, you containerize it.

Now, you can run the same app:

- on your teammate's machine
- on a server in the data center
- on the cloud (AWS, Azure, GCP)

... with the exact same behavior.

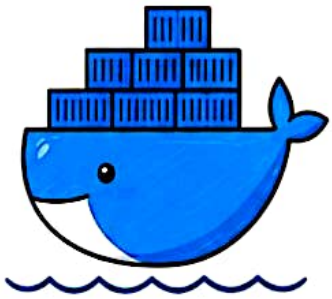


Docker makes applications portable, reliable and easy to manage.

02 OF 10

DOCKER ARCHITECTURE

Docker follows a client-server architecture. The Docker client communicates with the Docker daemon, which manages images, containers, networks and volumes.



Key Idea

You run Docker commands, the daemon does the heavy lifting.

ARCHITECTURE OVERVIEW

CLIENT

Docker CLI

You run commands like:

```
$ docker run
$ docker ps
$ docker build
$ docker pull
$ docker ...
```

REST API

Over Unix socket or TCP

DOCKER DAEMON

Docker Daemon (dockerd)

Listens for Docker API requests and manages Docker objects.

DOCKER OBJECTS

Images
Read-only templates used to create containers.

Containers
Runnable instances of images.

Networks
Enable communication between containers.

Volumes
Persistent storage for containers.

REGISTRY

Docker Hub (or Private Registry)

Store and distribute Docker images.

HOW IT WORKS

1. You run a Docker command using the CLI.
2. CLI sends the request to the Docker daemon.
3. Daemon processes the request.
4. Daemon manages Docker objects (images, containers, networks, volumes).
5. Results are returned to the CLI.

COMMAND EXAMPLE FLOW

```
$ docker run nginx
```

↓ CLI sends request to daemon

↓ Daemon pulls image (if not available)

↓ Daemon creates and starts container

↓ Container running

DAEMON LOCATION

Linux
`/var/run/docker.sock`
(Unix Socket)

Windows
`npipe:///./pipe/docker_engine`
(Named Pipe)

Mac
`/var/run/docker.sock`
(Unix Socket via VM)

KEY POINTS

- Docker is a client-server application.
- Daemon runs in the background.
- You can manage Docker locally or on a remote server.
- All Docker components (images, containers, networks, volumes) are managed by the daemon.

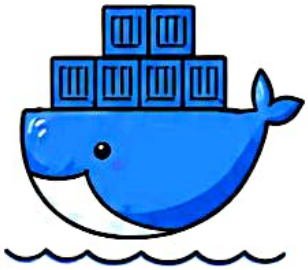
Remember: You talk to the Docker daemon, not directly to the containers.

Pro Tip: Use `docker context` to switch between different Docker environments.

03 OF 10

IMAGES & CONTAINERS

Images are read-only templates. Containers are running instances of images.



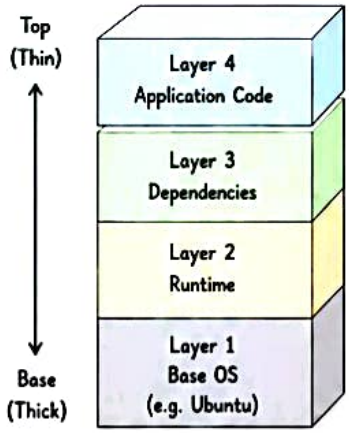
Core Idea

Build once, Run anywhere with the same consistency.

IMAGE VS CONTAINER

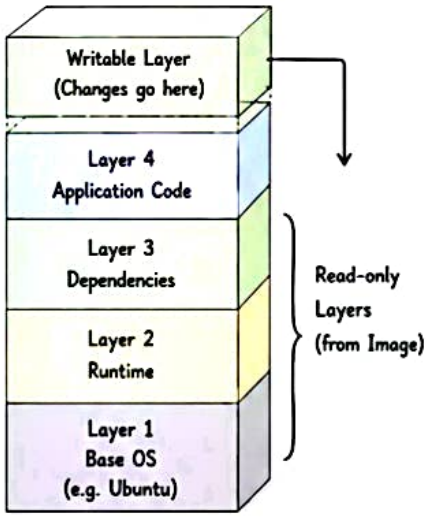
IMAGE	CONTAINER
Read-only template	Runnable instance
Built from Dockerfile	Created from an image
Consists of layers	Adds a thin writable layer on top
Can't be changed	Can be started, stopped, deleted
Stored in Registry	Exists on local system

IMAGE LAYERS

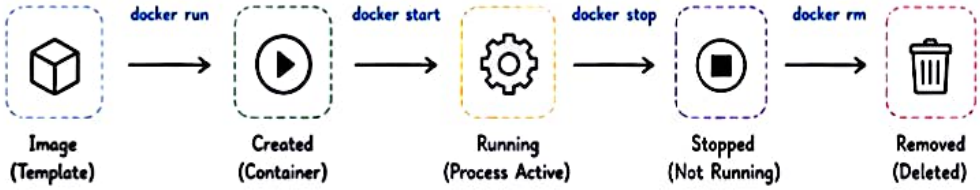


Layers are cached and reused. Only changes are rebuilt.

CONTAINER LAYER (WRITABLE)



CONTAINER LIFECYCLE



A container goes through its lifecycle. You can start, stop, restart and remove it anytime.

KEY POINTS

- Images are immutable.
- Containers are lightweight and isolated.
- Multiple containers can run from the same image.
- Deleting a container does not remove the image.

COMMON COMMANDS

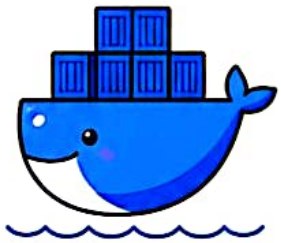
Pull Image	Run Container	List Containers	List All Containers	Stop Container	Remove Container	Remove Image
Download an image	Create and start a container	Show running containers	Show all containers (running + stopped)	Stop a running container	Remove a container	Remove an image
`docker pull nginx`	`docker run -d -p 8080:80 nginx`	`docker ps`	`docker ps -a`	`docker stop <id>`	`docker rm <id>`	`docker rmi <image>`

Pro Tip: Use images for distribution. Use containers for execution.

04 OF 10

DOCKER CLI
ESSENTIAL COMMANDS

Docker CLI (Command Line Interface) is used to manage Docker objects.



REMEMBER

Most Docker commands follow the pattern:

`docker <command> [options] [arguments]`

WHERE IS CLI USED?

- Local Machine
- Remote Server
- CI/CD Pipeline
- Cloud Environment

QUICK SYNTAX

```
docker <command> [options] [arguments]
```

COMMON DOCKER COMMANDS

CATEGORY	COMMAND	DESCRIPTION	EXAMPLE
CONTAINER	run	Create and start a new container	`docker run -d -p 8080:80 nginx`
	ps	List running containers	`docker ps`
	ps -a	List all containers (running + stopped)	`docker ps -a`
	start	Start a stopped container	`docker start <container_id>`
	stop	Stop a running container	`docker stop <container_id>`
	restart	Restart a container	`docker restart <container_id>`
	rm	Remove a stopped container	`docker rm <container_id>`
IMAGE	images	List all images	`docker images`
	pull	Download an image	`docker pull nginx`
	rmi	Remove an image	`docker rmi <image_id>`
	build	Build an image from Dockerfile	`docker build -t myapp .`
OTHER	logs	View container logs	`docker logs <container_id>`
	exec	Run command inside a container	`docker exec -it <id> bash`
	inspect	Detailed info about Docker objects	`docker inspect <container_id>`
	cp	Copy files/folders between container and host	`docker cp <id>:/app .`
	system prune	Remove unused data (containers, images, networks)	`docker system prune -a`

PRACTICAL WORKFLOW EXAMPLE

1. Pull Image — `docker pull nginx`
2. Run Container — `docker run -d -p 8080:80 nginx`
3. Check Running Containers — `docker ps`
4. View Logs — `docker logs <id>`
5. Stop Container — `docker stop <id>`
6. Remove Container — `docker rm <id>`

COMMON OPTIONS

-d	Run container in detached mode
-it	Interactive mode (terminal)
-p	Publish port (host:container)
-v	Mount volume
--name	Assign a name to container
--rm	Remove container when it exits

PRO TIPS

- Use tab auto-completion to save time.
- Use `docker ps -a` to see all containers.
- Use `docker system prune -a` to clean up unused resources.
- Logs are your best friend for debugging.

COMMAND HELP

```
docker --help
docker <command> --help
```

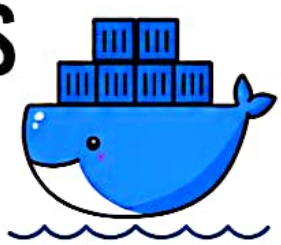
Example:
`docker run --help`

Master the basics, and Docker becomes your superpower.

05 OF 10

DOCKERFILE & BUILDS

A Dockerfile is a text file that contains instructions to build a Docker image.



Key Idea

Write once, build anywhere, run everywhere.

DOCKERFILE INSTRUCTIONS

INSTRUCTION	PURPOSE	EXAMPLE
FROM	Base image for the build	`FROM python:3.11-slim`
RUN	Run commands in the image	`RUN apt-get update && apt-get install -y \ curl`
COPY	Copy files/folders from host to image	`COPY app/ /app/`
ADD	Copy files and also download from URL, extract archives	`ADD https://example.com/file.tar.gz /tmp/`
WORKDIR	Set working directory for subsequent instructions	`WORKDIR /app`
EXPOSE	Document the port the container listens on	`EXPOSE 8080`
ENV	Set environment variables	`ENV FLASK_ENV=production`
CMD	Default command to run when container starts	`CMD ["python", "app.py"]`
ENTRYPOINT	Configure a container that will run as an executable	`ENTRYPOINT ["python", "app.py"]`
VOLUME	Create a mount point for volumes	`VOLUME ["/data"]`
ARG	Define build-time variables	`ARG VERSION=1.0`
USER	Set the username/UID to run subsequent instructions	`USER appuser`
HEALTHCHECK	Check container health on runtime	`HEALTHCHECK CMD curl -f http://localhost:8080/health || exit 1`

BUILD CONTEXT

The build context is the set of files sent to the Docker daemon during the build.

Project Directory

```
myapp/
├── Dockerfile
├── app/
│   ├── main.py
│   └── utils.py
├── requirements.txt
├── .dockerignore
└── data/
```

docker build → Docker Daemon

Only files inside the build context are available to the Docker daemon.

.DOCKERIGNORE

Exclude files and folders from the build context.

```
__pycache__/
*.pyc
*.log
.git/
node_modules/
data/
.env
Dockerfile*
```

Keeps images small and builds fast!

EXAMPLE DOCKERFILE

```
1  FROM python:3.11-slim
2  WORKDIR /app
3  COPY requirements.txt .
4  RUN pip install --no-cache-dir -r requirements.txt
5  COPY app/ ./app/
6  EXPOSE 8080
7  ENV FLASK_ENV=production
8  CMD ["python", "app/main.py"]
```

BUILD PROCESS (Layer by Layer)

`FROM python:3.11-slim`	Layer 1
`WORKDIR /app`	Layer 2
`COPY requirements.txt .`	Layer 3
`RUN pip install ...`	Layer 4
`COPY app/ ./app/`	Layer 5
`EXPOSE 8080`	Layer 6
`CMD ["python", "app/main.py"]`	Layer 7

CMD vs ENTRYPOINT

CMD

- Provides default command and arguments
- Can be overridden when running container

```
docker run myimage python app.py
```

ENTRYPOINT

- Sets the main executable
- Arguments can be passed or overridden

```
docker run myimage --debug
```

BUILD COMMAND

```
$ docker build -t myapp:1.0 .
```

- `docker build` — Build command
- `-t myapp:1.0` — Tag name (image name:tag)
- `.` — Build context (current directory)

Use meaningful tags!
Example: `myapp:1.0` , `myapp:latest`

LAYER CACHING

Step 1	Build →	Step 1 — ✓ Cached
Step 2		Step 2 — ✓ Cached
Step 3		Step 3 — ✕ Rebuild
Step 4		Step 4 — ✓ Cached

Docker reuses unchanged layers, making builds much faster.

BEST PRACTICES

- Use official base images.
- Keep images small.
- Leverage layer caching.
- Use .dockerignore.
- Run as non-root user.
- Scan images for vulnerabilities.

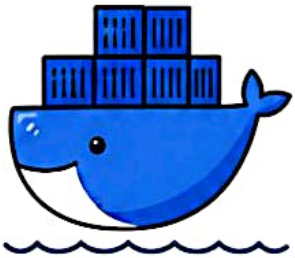
Write a good Dockerfile once, build reliably, run anywhere!

06 OF 10

DOCKER NETWORKING

Connect containers to the world and to each other.

Docker provides multiple networking drivers to enable communication between containers, the host and external networks.



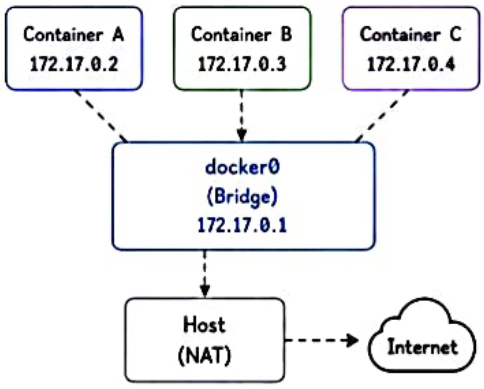
KEY IDEA

Containers have their own network namespace and IP address.

NETWORK DRIVERS

DRIVER	DESCRIPTION	USE CASE
bridge	Default network. Containers connect via virtual bridge (NAT to host).	Standalone containers on single host.
host	Container shares the host's network namespace.	High performance applications. (No network isolation)
none	No networking.	Highly isolated containers.
custom (user-defined)	Create custom networks (bridge) with better control, DNS and isolation.	Multi-container apps with communication needs.
overlay (swarm)	Connect containers across multiple Docker hosts.	Docker Swarm multi-host networking.

DEFAULT BRIDGE NETWORK (bridge)

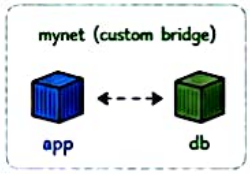


- Containers get IP from the bridge subnet.
- Outgoing traffic is NATed through the host.
- Containers on the same bridge can communicate.

CREATE A CUSTOM NETWORK

```
$ docker network create mynet
```

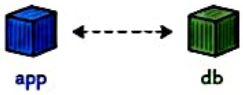
- Containers connected to the same custom network can communicate using container name as hostname.
- Provides automatic DNS resolution.




```
# Run containers in custom network
$ docker run -d --name app --network mynet nginx
$ docker run -d --name db --network mynet mysql
```

CONTAINER COMMUNICATION

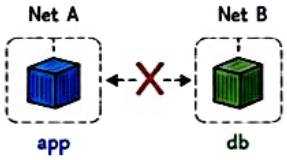
Same Network



They can talk using container name or IP.

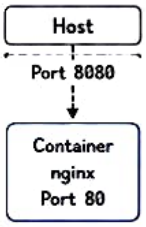
```
Inside app container:
ping db
curl http://db:3306
```

Different Networks



Containers on different networks cannot communicate by default. (Use network connect or routes)

PORT MAPPING (PUBLISHING PORTS)




```
$ docker run -d -p 8080:80 nginx
```

Host Port 8080 — Container Port 80

- Access: http://localhost:8080
- Format: -p <host_port>:<container_port>
- Use 0.0.0.0:8080:80 to bind on all interfaces.

HOST NETWORK

```
$ docker run -d --network host nginx
```

Host
IP: 192.168.1.10
Port: 80
Container (nginx)

- Container shares the host's network namespace.
- No port mapping needed.
- Faster, but no network isolation.
- Works only on Linux.

CHECK NETWORKS

```
$ docker network ls
$ docker network inspect mynet
$ docker network inspect bridge
```

TROUBLESHOOTING TIPS

- Use docker ps to get container IP.
- Use docker inspect <container>.
- Check firewall rules on host.
- Ensure containers are on same network.
- Use ping, curl, telnet for testing.

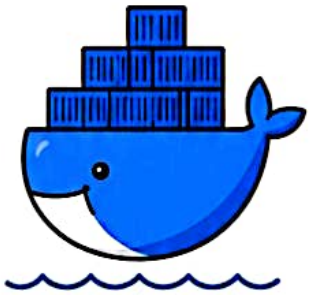
Choose the right network for the right use case.
Good networking = reliable, secure and scalable applications!

07 OF 10

DOCKER STORAGE

Persist data beyond the container lifecycle.

Containers are ephemeral. When a container is removed, its writable layer (data) is lost. Docker storage solutions help you persist and share data.



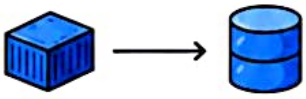
Key Idea

Use volumes or bind mounts for data that must survive container restarts or deletions.

TYPES OF DOCKER STORAGE

1 VOLUME (Managed by Docker)

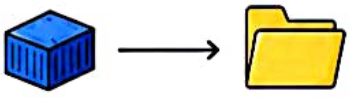
- Stored in Docker's storage directory.
- Managed by Docker.
- Best for data persistence.
- Can be shared between containers.



Use cases
- Database data
- Application data
- Persistent logs

2 BIND MOUNT (Managed by Host)

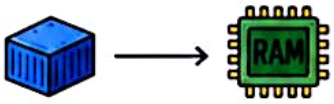
- Map a file or directory from host to container.
- Changes on host reflect inside container.
- More control, but less portable.



Use cases
- Source code
- Configuration files
- Development (live reload)

3 TMPFS MOUNT (In-Memory)

- Stored in host memory.
- Very fast but non-persistent.
- Data is lost when container stops.

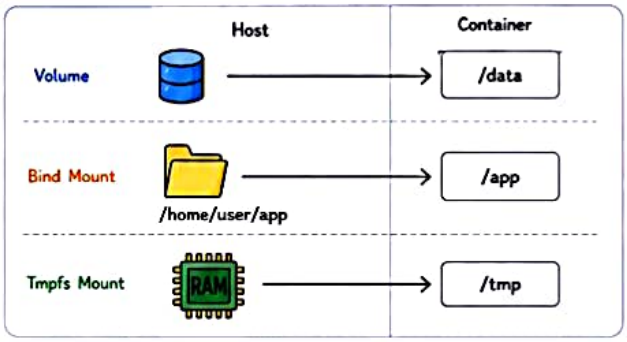


Use cases
- Temporary data
- Sensitive data (not written to disk)
- Cache

VOLUME vs BIND MOUNT vs TMPFS

Feature	VOLUME	BIND MOUNT	TMPFS MOUNT
Managed by	Docker	Host	Host (Memory)
Location	Docker storage (/var/lib/docker/volumes)	Any path on host	Host RAM
Persistence	Yes	Yes	No (lost on stop)
Performance	Good	Depends on disk	Very Fast
Portability	High	Lower	High
Best for	Production, databases	Development, configs, source code	Temp data, caching

WHERE DATA LIVES?



COMMON COMMANDS

VOLUMES

```
# Create a volume
docker volume create mydata

# List volumes
docker volume ls

# Inspect a volume
docker volume inspect mydata

# Remove a volume
docker volume rm mydata
```

BIND MOUNT

```
docker run -d \
-v /host/path:/container/path \
--name mycontainer nginx

Example:

docker run -d \
-v /home/user/app:/usr/src/app \
--name myapp node
```

TMPFS MOUNT

```
docker run -d \
--tmpfs /tmp:size=100m \
--name mycontainer nginx

Options:

size=100m  (max size)
mode=1777 (permissions)
```

MOUNT IN DOCKER COMPOSE

```
services:
  app:
    image: myapp
    volumes:
      - mydata:/data          # volume
      - ./code:/app           # bind mount
    tmpfs:
      - /tmp                  # tmpfs mount
volumes:
  mydata:
```

PERMISSIONS NOTES

- Bind mounts use host file permissions.
- Volume permissions are managed by Docker.
- Use 'chown' or 'chmod' inside container if needed.

VOLUME LOCATION (default)

```
Linux: /var/lib/docker/volumes/
Windows: C:\ProgramData\Docker\volumes\
macOS: /var/lib/docker/volumes/
```

BACKUP & MIGRATION

- Backup volumes: use docker run with -v volume:/data and tar.
- Bind mounts: backup host directory directly.

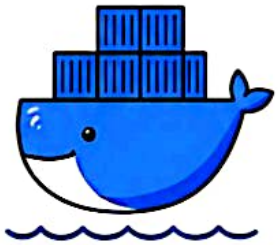
Always choose the right storage for the right need.
Good storage design = reliable and maintainable applications!

08 OF 10

DOCKER COMPOSE

Define and run multi-container applications easily.

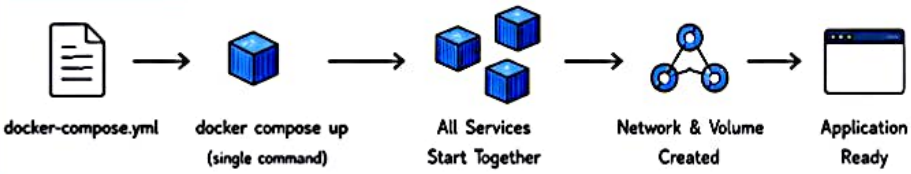
Docker Compose uses a YAML file to configure your application's services, networks and volumes.
Then with one command, everything is up and running.



WHY COMPOSE?

- Define everything in code
- Easy to version & share
- Start everything together
- Manage multi-container apps

HOW IT WORKS



TYPICAL USE CASES

- Web app + Database
- API + Cache + Database
- Microservices locally
- Dev/Test environments

EXAMPLE: docker-compose.yml

```yaml
version: '3.8'
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    depends_on:
      - app
    networks:
      - mynet
  app:
    build: ./app
    ports:
      - "5000:5000"
    environment:
      - DB_HOST=db
    depends_on:
      - db
    networks:
      - mynet
  db:
    image: postgres:14
    environment:
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=mydb
    volumes:
      - db_data:/var/lib/postgresql/data
    networks:
      - mynet
volumes:
  db_data:
networks:
  mynet:
    driver: bridge
```

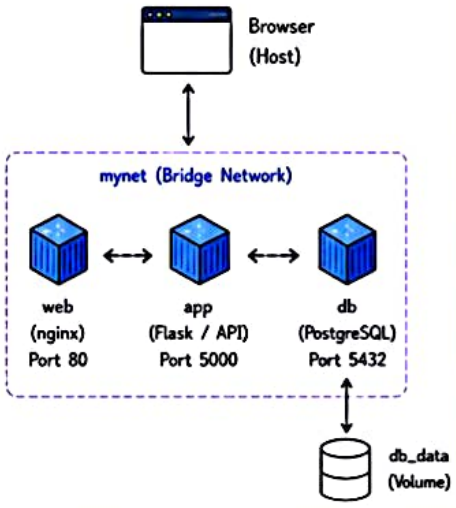
COMPONENTS EXPLAINED

Key	Description
version	Compose file version
services	List of containers (services)
image / build	Image to use or path to build
ports	Map host port to container port
environment	Environment variables
depends_on	Start order of services
volumes	Persistent data storage
networks	Define custom networks

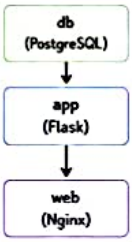
COMMON COMMANDS

docker compose up	Start all services
docker compose up -d	Start in background
docker compose down	Stop and remove containers, networks, volumes
docker compose ps	List running services
docker compose logs	View logs of all services
docker compose logs <service>	View logs of a specific service
docker compose build	Build images
docker compose exec <service> sh	Open shell in a service
docker compose restart	Restart all services

ARCHITECTURE (EXAMPLE ABOVE)



DEPENDENCY (depends_on)



db starts first,
then app,
then web.

It does NOT wait for service to be 'ready', only for it to start.

VOLUMES IN COMPOSE

Named Volume

Managed by Docker

```
db_data:/var/lib/
postgresql/data
```

Bind Mount

Map host path to container

```
./data:/app/data
```

ENVIRONMENT VARIABLES

.env

```
DB_USER=postgres
DB_PASSWORD=pass
DB_NAME=mydb
```

Use in compose file:

```
environment:
  - DB_USER=${DB_USER}
  - DB_PASSWORD=${DB_PASSWORD}
  - DB_NAME=${DB_NAME}
```

BEST PRACTICES

- Keep compose.yml in project root.
- Use .env file for sensitive values.
- Use named volumes for data persistence.
- Use custom networks for isolation.
- Use healthcheck for critical services.

One command to rule them all!

```
$ docker compose up -d
```

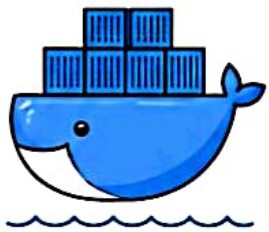
Simpler setups,
faster development!

09 OF 10

DOCKER HUB

Share, discover and use Docker images.

Docker Hub is a cloud-based registry service provided by Docker for finding, sharing and storing container images.



KEY IDEA

- Find official and community images
- Store your own images
- Pull images from anywhere
- Push images to share

DOCKER HUB FEATURES

Official Images
High-quality images maintained by Docker and trusted partners.

Community Images
Images created and shared by the community.

Push & Pull
Push your images to Docker Hub and pull any image from anywhere.

Repositories
Organize images in repositories (public or private).

Tags
Use tags to manage different versions of images.

Access Control
Set repository visibility (public/private) and manage team access.

COMMON COMMANDS

COMMAND	DESCRIPTION
`docker login`	Login to Docker Hub
`docker logout`	Logout from Docker Hub
`docker search <image>`	Search for images on Docker Hub
`docker pull <image>:<tag>`	Pull image from Docker Hub
`docker push <image>:<tag>`	Push image to Docker Hub
`docker images`	List local images
`docker tag <src> <user/repo>:<tag>`	Tag an image for push
`docker rmi <image>`	Remove local image

WORKFLOW

Search Image
`docker search nginx`

↓

Pull Image
`docker pull nginx:latest`

↓

Run Container
`docker run -d -p 80:80 nginx`

↓

Tag Image
`docker tag nginx:latest myuser/nginx:v1`

↓

Push Image
`docker push myuser/nginx:v1`

IMAGE NAME FORMAT

```
[registry/] [username/] repository[:tag]
```

registry : Optional registry address (default: Docker Hub)
username : Docker Hub username or org (default: library for official images)
repository : Image name
tag : Specific version (default: latest)

Example: `docker.io/myuser/myapp:v1`

PUBLIC vs PRIVATE REPOSITORY

PUBLIC
- Visible to everyone
- Can be pulled by anyone
- Good for open source
- Free on Docker Hub

PRIVATE
- Visible only to you or your team
- Requires login to pull
- Good for private projects
- Some limits on free plan

PRACTICAL EXAMPLE

1. **Login** — `docker login` — Login with your Docker Hub account.
2. **Search Image** — `docker search nginx` — Find images on Docker Hub.
3. **Pull Image** — `docker pull nginx:latest` — Download image to your machine.
4. **Run Container** — `docker run -d -p 8080:80 nginx` — Run container from the image.
5. **Tag Image** — `docker tag nginx:latest myuser/nginx:v1` — Tag the image for your repository.
6. **Push Image** — `docker push myuser/nginx:v1` — Push image to Docker Hub.

DOCKERFILE + DOCKER HUB WORKFLOW

Write Dockerfile → Build Image (`docker build -t myapp:v1 .`) → Tag Image (`docker tag myapp:v1 myuser/myapp:v1`) → Push Image (`docker push myuser/myapp:v1`) → Image available on Docker Hub

HELPFUL TIPS

- Use meaningful repository names.
- Always use tags (don't rely on latest).
- Keep images small (use .dockerignore and multi-stage builds).
- Never push sensitive data or credentials.
- Regularly clean unused images locally.

DOCKER HUB LIMITS (FREE PLAN)

Feature	Free Plan Limit
Public Repositories	Unlimited
Private Repositories	1
Team Members	1
Image Pulls	Limited (rate limited)
Builds (Docker Build Cloud)	Limited

QUICK CHEAT SHEET

```
docker login                              # Login to Docker Hub
docker search <image>                     # Search image
docker pull <image>:<tag>                 # Pull image
docker images                             # List images
docker tag <src> <user/repo>:<tag>        # Tag image
docker push <user/repo>:<tag>             # Push image
docker logout                             # Logout
```

IMPORTANT NOTES

- Docker Hub is the default registry for Docker.
- Official images are trusted and well-maintained.
- Private repos require authentication.
- Images are versioned using tags.
- Use docker login before pushing/pulling private images.

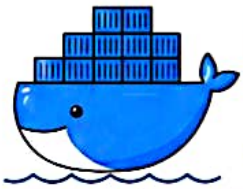
Docker Hub makes collaboration easy.
Build once, share everywhere!

10 OF 10

DOCKER – SUMMARY & NEXT STEPS

You've made it! 🎉

You now know the essentials to build, run and manage containers with Docker.



Key Takeaway

Docker helps you package applications with all their dependencies and run them anywhere consistently.

DOCKER ECOSYSTEM

Docker Engine
Builds and runs containers.

Docker Images
Templates used to create containers.

Docker Containers
Running instances of images.

Docker Networks
Allows communication between containers.

Docker Volumes
Persist and share data between containers.

Docker Hub
Cloud registry to find, share and store images.

DOCKER COMMAND CHEAT SHEET

COMMAND	PURPOSE
`docker build -t image:tag .`	Build an image from Dockerfile
`docker run -d --name name image`	Run container in detached mode
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker stop <id/name>`	Stop a container
`docker start <id/name>`	Start a stopped container
`docker rm <id/name>`	Remove a container
`docker rmi <image>`	Remove an image
`docker images`	List images
`docker logs <id/name>`	View container logs
`docker exec -it <id/name> bash`	Access container shell
`docker --version`	Check Docker version

DOCKERFILE QUICK RECAP

FROM	Base image
WORKDIR	Set working directory
COPY	Copy files to image
RUN	Run commands
EXPOSE	Expose port
ENV	Set environment variable
CMD	Default command
ENTRYPOINT	Main executable
VOLUME	Create mount point
ARG	Build-time variable
USER	Set user
HEALTHCHECK	Health check for container

DOCKER COMPOSE RECAP

docker-compose.yml

```yaml
version: '3.8'
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    depends_on:
      - db
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
    volumes:
      - db_data:/var/lib/postgresql/data
volumes:
  db_data:
```

Common Compose Commands

`docker compose up`
Start services

`docker compose up -d`
Start in background

`docker compose down`
Stop and remove

`docker compose logs`
View logs

`docker compose build`
Build images

`docker compose exec <svc> sh`
Access service shell

PERSISTENCE OPTIONS

TYPE	MANAGED BY	PERSISTENT	USE CASE
Volume	Docker	✓ Yes	Databases, App data
Bind Mount	Host	✓ Yes	Source code, Configs
Tmpfs Mount	Memory	✗ No	Temp data, Cache

NETWORK DRIVERS

DRIVER	DESCRIPTION	USE CASE
bridge	Default network, NAT to host	Most containers
host	Use host network	High performance
none	No networking	Isolated containers
custom (user-defined)	Create custom network (bridge)	Multi-container apps
overlay	Multi-host networking (Swarm)	Docker Swarm

PORT MAPPING

Host Port 8080 → maps to → Container Port 80

```
docker run -d -p 8080:80 nginx
```

- 8080 → Host port
- 80 → Container port

WHERE DATA LIVES?

- Volume → Managed by Docker /var/lib/docker/volumes/...
- Bind Mount → On Host /path/on/host/...
- Tmpfs Mount → Host Memory (RAM) (lost on restart)

DEPENDENCY (depends_on)

db (Database)
↓
app (Backend)
↓
web (Frontend)

- Docker Compose starts services in order.
- It does NOT wait for a service to be ready. Use healthchecks for reliability.

BEST PRACTICES

- ✓ Use official base images.
- ✓ Keep images small.
- ✓ Use .dockerignore.
- ✓ Use named volumes for persistence.
- ✓ Don't run containers as root.
- ✓ Scan images for vulnerabilities.
- ✓ Use environment variables for configs and secrets.

COMMON FILES

- **Dockerfile** — Instructions to build image
- **docker-compose.yml** — Define and run multi-container apps
- **.dockerignore** — Exclude files from build context
- **.env** — Store environment variables

USE CASE EXAMPLES

- **Web Application** — Nginx + App + Database
- **API + Database** — Backend API with PostgreSQL
- **Microservices** — Multiple services in isolation
- **Dev/Test Environment** — Consistent across teammates

WHAT'S NEXT?

- Learn Docker Swarm (Orchestration)
- Kubernetes (Container Orchestration)
- CI/CD with Docker
- Docker Security
- Cloud Platforms (AWS/GCP/Azure)

Keep building. Keep learning. 🚀

Congratulations! You've completed the Docker Essentials.
Practice, build real projects and keep improving! 💪